

ЛАБОРАТОРНА РОБОТА № 4

4.1 Тема

Використання функцій та параметрів у мові програмування Python

4.2 Мета роботи

Ознайомитися з поняттям функцій у мові Python, навчитися визначати та викликати функції, передавати аргументи, використовувати параметри за замовчуванням і повертати значення з функцій.

х.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- призначення функцій у програмуванні;
- синтаксис визначення функцій у Python;
- способи передачі аргументів у функції;
- принцип роботи оператора ***return***;
- різницю між локальними та глобальними змінними.

2) уміти:

- визначати та викликати функції;
- передавати аргументи позиційно та за іменем;
- використовувати параметри за замовчуванням;
- повертати значення з функцій та обробляти їх;
- структурувати код за допомогою функцій для підвищення його читабельності.

3) набути навичок:

- створювати власні функції для розв'язання практичних завдань;
- застосовувати функції для обчислень та обробки даних;
- розробляти програму з модульною структурою;
- налагоджувати функції та перевіряти правильність їх роботи.

4.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

4.5 Теоретичні відомості

Функції у програмуванні: визначення та призначення

Функція – це іменований блок коду, який виконує певну задачу та може бути повторно використаний у програмі.

Вона дозволяє структурувати програму, роблячи її більш зрозумілою, компактною та легкою для супроводу.

Основні характеристики функції:

- має назву, за якою її можна викликати;
- може приймати аргументи (параметри) для обробки даних;
- може повертати результат обчислень;
- дозволяє повторно використовувати код.

Призначення функцій

Повторне використання коду:

- функції дозволяють один раз написати блок коду і використовувати його багаторазово;
- приклад: обчислення площі кола для різних радіусів без повторного написання формули.

Структурування та організація коду:

- великі програми легше читати, якщо логічно пов'язані дії оформлені як функції;
- це полегшує розробку модульного коду.

Спрощення налагодження та супроводу:

- помилки легше знайти у невеликому блоці коду (функції), ніж у великому монолітному коді;
- зміни в логіці можна вносити у функцію без впливу на інші частини програми.

Підвищення читабельності коду:

- назви функцій описують їх призначення: `calculate_area()` одразу показує, що функція обчислює площу.

Ізоляція логіки та полегшення тестування:

- кожна функція виконує обмежену задачу, що спрощує тестування і повторне використання.

Синтаксис функцій у Python

Оголошення функції

```
def назва_функції(параметри):
    """
    Документаційний рядок (docstring)
    Описує призначення функції та аргументи
    """
    тіло_функції
    return результат # необов'язково
```

Пояснення:

- **def** – ключове слово для оголошення функції;
- `назва_функції` – унікальна назва для виклику функції;
- параметри – змінні, що передаються функції (можуть бути відсутні);
- **return** – оператор, що повертає значення функції (не обов'язковий).

Аргументи та параметри функції

Позиційні аргументи

Значення передаються у порядку, у якому вони оголошені у функції.

```
def greet(name, age):
    print(f"Привіт, {name}! Тобі {age} років.")
greet("Оля", 20)
```

Ключові (іменовані) аргументи

Значення передаються за назвою параметра, порядок не важливий.

```
greet(age=20, name="Оля")
```

Параметри за замовчуванням

Значення параметра встановлюється за замовчуванням, якщо його не передано.

```
def greet(name, age=18):
    print(f"Привіт, {name}! Тобі {age} років.")
greet("Іван") # Використає age=18
```

Аргументи довільної кількості (*args, **kwargs)

*args – список позиційних аргументів;

**kwargs – словник а іменованих аргументів.

Повернення значень з функції

Оператор **return** використовується для передачі результату виклику функції.

Якщо **return** відсутній, функція повертає **None**.

```
def add(a, b):
    return a + b

result = add(5, 7)
print(result) # Виведе 12
```

Документаційний рядок (docstring)

Використовується для опису призначення функції, параметрів та результатів.

Розміщується в трьох лапках після оголошення функції.

Доступ до нього можна отримати через help().

```
def multiply(a, b):
    """Повертає добуток двох чисел a і b"""
    return a * b

help(multiply)
```

Рекурсія

Функція може викликати сама себе, це називається рекурсія.

Використовується для задач, які природно розбиваються на підзадачі, наприклад, факторіал, числа Фібоначчі.

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

Приклади застосування функцій

- математичні обчислення: площа кола, периметр прямокутника, факторіал;
- обробка даних: перевірка рядків, підрахунок статистики, сортування списків;
- взаємодія з користувачем: введення даних, валідація, форматування виводу;
- робота з файлами: читання, запис, обробка даних у текстових та JSON-файлах.

Резюме

Функції є основним інструментом модульного програмування.

Вони дозволяють:

- повторно використовувати код,
- структурувати програму,
- підвищувати читабельність,
- спрощувати тестування та супровід.

Правильне використання функцій формує логічне мислення та навички програмування, необхідні для ефективної роботи з Python.

4.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

4.7 Загальне завдання

Написати програму, яка реалізує функцію для обчислення за індивідуальним варіантом.

Функцію необхідно викликати для кількох різних значень та вивести результати на екран.

4.8 Індивідуальні завдання

- 1) Написати функцію для обчислення периметра прямокутника за довжиною та шириною.
- 2) Реалізувати функцію для обчислення факторіала числа.
- 3) Написати функцію для переведення температури з градусів Цельсія у Фаренгейти.
- 4) Створити функцію для переведення кілометрів у метри.
- 5) Реалізувати функцію для переведення годин у хвилини.
- 6) Написати функцію для обчислення середнього арифметичного кількох чисел.
- 7) Створити функцію для обчислення площі квадрата.
- 8) Реалізувати функцію для знаходження найбільшого з трьох чисел.
- 9) Написати функцію для переведення дюймів у сантиметри.
- 10) Створити функцію для підрахунку кількості голосних літер у рядку.

Нульовий варіант (приклад виконання)

Завдання:

Реалізувати функцію для обчислення площі кола за заданим радіусом і вивести результат на екран.

Приклад реалізації:

```
# Імпортуємо модуль math для використання числа π
import math

# Визначаємо функцію для обчислення площі кола
def calculate_circle_area(radius):
    """
    Функція обчислює площу кола за заданим радіусом.

    Аргументи:
    radius (float) - радіус кола

    Повертає:
    Площу кола (float)
    """
    area = math.pi * radius ** 2 # Формула площі кола: π * r^2
    return area # Повертаємо результат

# Основна частина програми
def main():
    # Запитуємо у користувача радіус кола
    r = float(input("Введіть радіус кола: "))

    # Викликаємо функцію та зберігаємо результат
    circle_area = calculate_circle_area(r)

    # Виводимо результат на екран
    print(f"Площа кола з радіусом {r} дорівнює {circle_area:.2f}")

# Перевіряємо, чи програма запущена напряму
if __name__ == "__main__":
    main()
```

Результат роботи:

```

PS E:\GoogleDrive\python-basic> e;; cd 'e:\GoogleDrive\python-basic'
; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-p
ython.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '482
31' '--' 'E:\GoogleDrive\python-basic\test.py'
Введіть радіус кола: 20
Площа кола з радіусом 20.0 дорівнює 1256.64
PS E:\GoogleDrive\python-basic>

```

Пояснення коду:

- 1) модуль `math` використовується для точного значення числа π ;
- 2) функція `calculate_circle_area` приймає один параметр – радіус, обчислює площу та повертає результат;
- 3) основна програма (`main`):
 - зчитує значення радіуса від користувача;
 - викликає функцію для обчислення площі;
 - виводить результат у зручному форматі (2 знаки після коми);
- 4) використання блоку `if __name__ == "__main__":`: дозволяє запускати код тільки при прямому виконанні файлу.

4.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

4.10 Контрольні питання

- 1) Як визначити функцію в Python?
- 2) Що таке параметри та аргументи функції?
- 3) Яке призначення оператора return?
- 4) Як передавати аргументи за іменами?
- 5) Що таке рекурсія та де її доцільно використовувати?

4.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>